

Agentic AI: Foundations and Open-Source Practice

Day 2 Study Material

Core Components of an Agent and Reasoning Patterns

1. Session Overview

Unit	Unit 1: Foundations of AI Agents
Sub-topics	1.3 Core components: LLM, tools, orchestration loop, memory, system prompt 1.4 Reasoning patterns: Chain-of-Thought, ReAct, Plan-and-Execute, Reflection
Course Outcome	CO1
Builds on	Day 1: Agent = LLM + Tools + Loop; the agent you ran in the lab

Learning Objectives

By the end of Day 2, you will be able to:

1. Describe the five core components of an AI agent and the role each one plays.
2. Explain what makes a good tool description and why the system prompt is part of the engineering.
3. Explain the orchestration loop, its stopping conditions, and its failure modes.
4. Distinguish short-term from long-term memory in an agent.
5. Explain Chain-of-Thought prompting and write a CoT prompt.
6. Trace a ReAct cycle of Thought, Action, and Observation for a multi-step question.
7. Compare ReAct, Plan-and-Execute, and Reflection, and choose a suitable pattern for a task.

2. Video Lectures

Watch Videos 1 and 2 before class. Videos 3 and 4 deepen the reasoning-pattern topic, and Video 5 is a short revision of the components.

No.	Title and Link	When and Why
1	Chain-of-Thought (CoT) Prompting Explained https://www.youtube.com/watch?v=4aZFbuOUwmA	Watch before class. Explains why asking a model to reason step by step improves its answers. Covers sub-topic 1.4.
2	ReAct AI Agents, clearly explained! https://www.youtube.com/watch?v=vFdlrZyKEwQ	Watch before class. Walks through the Thought, Action, Observation cycle that your Day 1 agent already performs.
3	AI Agentic Design Patterns: ReAct Explained https://www.youtube.com/watch?v=WBgl9ce_7wM	Recommended. Presents ReAct as a design pattern and compares it with plain prompting.
4	What's next for AI agentic workflows ft. Andrew Ng https://www.youtube.com/watch?v=sal78ACtGTc	Recommended (about 14 minutes). Introduces reflection, tool use, planning, and multi-agent collaboration as the four agentic design patterns.

No.	Title and Link	When and Why
5	Chain of Thought (CoT) Prompts Explained with Examples https://www.youtube.com/watch?v=LdFB4J9SD9E	Optional. More worked CoT examples, with no coding.
6	What are AI Agents? (IBM Technology) https://www.youtube.com/watch?v=F8NKVhkZZWI	Revision of Day 1. Useful again here for the components: LLM, tools, and memory.

If a link does not open, search the exact video title on YouTube.

3. Lecture Notes: 1.3 Core Components of an Agent

On Day 1 you learned the short formula: **Agent = LLM + Tools + Loop**. That formula is enough to build a first agent. To build a useful one, two more components are needed: **memory** and the **system prompt**.

3.1 The Five Components at a Glance

Component	Role	In your Day 1 code
LLM	The reasoning engine. Decides what to do next.	<code>client.chat.completions.create(...)</code>
Tools	The hands. Actions the agent can request.	<code>get_course_fee</code> and <code>calculator</code> in <code>tools.py</code>
Orchestration loop	The driver. Runs tools and calls the LLM again until the task is done.	the for loop in <code>agent.py</code>
Memory	What the agent knows about the task so far.	the growing <code>messages</code> list
System prompt	The rules of behaviour, written by the developer.	<code>SYSTEM_PROMPT</code> in <code>agent.py</code>

3.2 The LLM: the Reasoning Engine

The LLM is consulted once per step. At each call it reads the whole conversation so far and produces one of two things: a **tool call** (a request to act) or **plain text** (the final answer).

What the LLM is responsible for:

- Understanding the goal stated by the user.
- Deciding whether a tool is needed, and which one.
- Filling in the tool arguments correctly.
- Interpreting the observation that comes back.
- Deciding that the task is complete.

What it is **not** responsible for: running the tool, storing data, or enforcing rules. Those belong to your program.

Choosing a model matters. A model that is excellent at writing essays may be poor at tool calling. For agents, the qualities to look for are instruction following, reliable structured output, a long enough context window, and acceptable latency and cost. This is covered in depth in Unit 2.

3.3 Tools: the Hands of the Agent

A tool is an ordinary function in your program that the LLM is allowed to request. Each tool is described to the model with a name, a description, and a list of parameters.

Type of tool	Examples
Read tools	Database lookup, document search, web search, file reading, API calls that only fetch data
Compute tools	Calculator, code execution, unit conversion, date arithmetic
Write tools	Sending email, updating a record, creating a ticket, making a payment
Human tools	Asking the user a question, or requesting approval before an action

Two rules for tools:

- **The description is a prompt, not a comment.** It is the only information the model has when choosing between tools. "Get the fee in rupees for a single course code, for example CS101" works far better than "Gets data".
- **Give the smallest set of tools that does the job.** Every extra tool increases the chance of a wrong choice, and every write tool increases the damage a mistake can cause. Write tools should be read-only wherever possible, or require human approval.

3.4 The Orchestration Loop

The loop is plain code that you control. Its structure is always the same:

```
messages = [system prompt, user question]

repeat up to max_steps times:
    reply = LLM(messages, tools)          # REASON
    if reply has no tool call:
        return reply                      # FINISH
    for each tool call in reply:
        result = run_the_tool(...)        # ACT
        add result to messages            # OBSERVE

return "stopped: step limit reached"
```

Design decision	Why it matters
Stopping when no tool is requested	The model, not your code, signals that the task is finished. This is what makes it an agent rather than a fixed sequence.
A maximum step limit	Prevents infinite loops and runaway cost. Without it, a confused model can call tools forever.
Appending every result to messages	The model is stateless, so anything not in the list is forgotten. The list is the agent's working memory.
Catching tool errors and returning them as text	An error message lets the model correct itself. An uncaught exception ends the run.

Design decision	Why it matters
Logging each step	The printed trace is how you debug an agent. Without it, a wrong answer gives you nothing to inspect.

Common loop failure modes: repeating the same tool call forever, calling a tool with wrong or empty arguments, ignoring the tools and guessing, stopping too early with an incomplete answer, and growing the message list until it exceeds the context window.

3.5 Memory

An LLM remembers nothing between calls. Any memory an agent appears to have is memory your program supplies.

Kind of memory	What it is
Short-term (working) memory	The messages list for the current task: the system prompt, the user's question, the tool calls, and the observations. It lives only as long as the run.
Conversation memory	The history of earlier turns with the same user, so "and what about DS303?" makes sense. Usually the same list, kept across questions.
Long-term memory	Facts stored outside the conversation, in a file or database, and retrieved when relevant: a student's preferences, past decisions, or project details.
Retrieved knowledge	Documents fetched for this question from a knowledge base. This is RAG, covered in Unit 4.

The practical problem is size. Context windows are limited and every token costs time and money, so long conversations must be managed by trimming old messages, summarizing them, or storing them outside the model and retrieving only what matters. You will implement real memory with LangGraph checkpoints in Unit 3.

3.6 The System Prompt

The system prompt is the developer's instruction to the model, sent before the user's question and kept at the top of the conversation. It is the cheapest and fastest way to change an agent's behaviour, and it is part of the engineering, not decoration.

A good agent system prompt usually states:

- **Role and scope:** who the agent is and what it does not handle.
- **Tool policy:** when to use each tool, and what must never be answered from memory.
- **Facts the agent needs:** small, stable information such as the list of valid course codes.
- **Output format:** how the answer should be presented.
- **Limits:** what to do when information is missing, and when to ask a human.

Your Day 1 prompt, line by line:

Phrase	What it is doing
You are a college fee assistant	Role and scope

Phrase	What it is doing
Never guess a fee: always use get_course_fee	Tool policy, and a direct fix for hallucination
Use calculator for any arithmetic	Moves unreliable arithmetic into code
Available course codes: CS101, AI202, DS303	A small, stable fact the agent cannot look up
If no tool is needed, answer directly	Prevents needless tool calls

Remember that the system prompt is advice, not a guarantee. A model can ignore it, and a user can try to override it through prompt injection. Real safety comes from limiting what the tools can do, which is the subject of Unit 8.

3.7 How the Components Fit Together

```

      SYSTEM PROMPT (rules, written once)
      |
user question ---> [ MEMORY: messages list ] <-----+
      |
      v
      [ L L M ] -- decides --> tool call
      |
no tool call
      |
      v
      final answer      observation ---+
                          (the LOOP repeats this cycle)

```

4. Lecture Notes: 1.4 Reasoning Patterns

A reasoning pattern is the shape of the thinking you ask the model to do. The same model, the same tools, and the same question can give very different results depending on the pattern used.

4.1 Chain-of-Thought (CoT)

Idea: ask the model to work through the problem step by step before answering, instead of jumping to the result. Introduced by Wei and colleagues (NeurIPS 2022).

Without CoT:

```

Q: A student pays Rs. 12,000 for CS101 and Rs. 18,000 for AI202,
   and receives a 10% scholarship. What is the total fee?
A: Rs. 28,000      <-- wrong

```

With CoT (adding "Let's think step by step"):

```

Step 1: Total before scholarship = 12,000 + 18,000 = 30,000
Step 2: Scholarship = 10% of 30,000 = 3,000
Step 3: Total after scholarship = 30,000 - 3,000 = 27,000
Answer: Rs. 27,000      <-- correct

```

Variant	How it works
Zero-shot CoT	Add an instruction such as "Think step by step" or "Show your reasoning before the final answer".
Few-shot CoT	Give one or two solved examples that show the reasoning, so the model imitates the style.
Self-consistency	Generate several reasoning paths and take the answer that appears most often.

Why it works: each written step becomes part of the input for the next token, so the model has more to work with than if it had to produce the answer in one jump.

Limits: CoT reasons only over what the model already knows plus what is in the prompt. If the fee data is not there, it will reason beautifully over invented numbers. It also makes answers longer, slower, and more expensive, and the written reasoning is not always the true cause of the answer.

4.2 ReAct (Reasoning and Acting)

Idea: interleave reasoning with actions. The model alternates between thinking, using a tool, and reading the result. Introduced by Yao and colleagues (ICLR 2023). The cycle is

Thought, Action, Observation, repeated until the answer is ready.

Question: What is the total fee for CS101 and AI202 after a 10% scholarship?

Thought: I need the fee for CS101.
 Action: `get_course_fee("CS101")`
 Observation: 12000

Thought: Now I need the fee for AI202.
 Action: `get_course_fee("AI202")`
 Observation: 18000

Thought: Add them and apply the 10% scholarship.
 Action: `calculator("(12000 + 18000) * 0.9")`
 Observation: 27000.0

Thought: I have the answer.
 Final Answer: The total fee is Rs. 27,000.

This is exactly what your Day 1 agent did. The printed step lines were its actions and observations; the thoughts were internal.

Property	Note
Relation to CoT	ReAct can be seen as Chain-of-Thought with actions added. Thinking alone becomes thinking plus checking.
Main benefit	Facts come from tools, not from memory, which greatly reduces hallucination.
Second benefit	The trace is readable, so you can see exactly where a wrong answer came from.
Main risks	It can repeat the same reasoning and action and fall into a loop; a wrong early observation can carry through the rest of the run.
Cost	One LLM call per step, so a four-step answer costs four calls.

4.3 Plan-and-Execute

Idea: make the model write a complete plan first, then carry out the steps, instead of deciding one step at a time.

```
PLAN (one LLM call):
1. Look up the fee for CS101
2. Look up the fee for AI202
3. Look up the fee for DS303
4. Add each pair and compare with 30000
5. Report the pairs that fit

EXECUTE: run steps 1 to 5, calling tools as required
REPLAN (if needed): if a step fails or the plan turns out to be wrong,
ask the model for a revised plan
```

Aspect	Note
Advantage	Fewer LLM calls, because the thinking is done once instead of at every step. Cheaper and often faster for long tasks.
Advantage	The plan can be shown to a human for approval before anything is executed.
Advantage	Keeps long tasks on track; a step-by-step agent can drift away from the original goal.
Disadvantage	A plan made before seeing any results can be wrong, so a replanning step is usually needed.
Variant	ReWOO plans all tool calls in advance and fills in the results afterwards, reducing the number of model calls further.

4.4 Reflection (Self-Critique)

Idea: after producing an answer, the model reviews its own work against the requirements, finds problems, and produces an improved version. The cycle is **generate, critique, revise**, repeated once or twice.

```
Draft: "You can take CS101 and AI202 for Rs. 30,000."
Critique: "Check the arithmetic: 12000 + 18000 = 30000, which is exactly
the budget, so it fits. But DS303 + CS101 = 27000 also fits
and was not mentioned. The answer is incomplete."
Revised: "Two options fit your budget of Rs. 30,000: CS101 + DS303 at
Rs. 27,000, and CS101 + AI202 at exactly Rs. 30,000."
```

Aspect	Note
Best used for	Writing, code, and any output where quality improves on a second pass.
Stronger when grounded	Critique based on something real (test results, a checklist, a tool output) works much better than critique based on opinion alone.
Disadvantage	Doubles or triples the cost, and a model often cannot see its own blind spots.
Risk	Endless revision. Fix the number of reflection rounds, usually one or two.
Related work	Reflexion adds memory of past mistakes so the agent avoids repeating them.

4.5 Comparison of the Four Patterns

	Chain-of-Thought	ReAct	Plan-and-Execute	Reflection
Core idea	Think step by step	Think, act, observe, repeat	Plan fully, then execute	Draft, critique, revise
Uses tools	No	Yes	Yes	Optional
Source of facts	Model's own knowledge	Tools and data	Tools and data	Whatever the draft used
LLM calls	One	One per step	One plan plus execution	Two or three per answer
Main strength	Better reasoning, no setup	Grounded and traceable	Efficient on long tasks	Higher quality output
Main weakness	Can reason over wrong facts	Loops, error propagation	Plan can be wrong	Costly; blind to own errors
Typical use	Maths, logic, analysis	Question answering with data	Multi-step projects	Writing and code review

4.6 Choosing a Pattern

If the task...	Use
is pure reasoning over information you already provide	Chain-of-Thought
needs live, private, or verifiable data	ReAct
has many steps and a predictable structure, or needs human approval before acting	Plan-and-Execute
produces an artifact whose quality matters (a report, an email, code)	Reflection, added on top of another pattern
is simple and single-step	None of these; a plain LLM call is enough

These patterns combine. A practical agent often plans first, uses ReAct for each step of the plan, and reflects once at the end.

5. Worked Example: One Question, Four Patterns

Question: "I can pay Rs. 30,000. Which two courses can I take together within this budget?" (Fees: CS101 = 12,000, AI202 = 18,000, DS303 = 15,000.)

Pattern	What happens
Chain-of-Thought	Reasons carefully about pairs and budgets, but has no fee data, so it invents figures. The reasoning is sound and the answer is unreliable.
ReAct	Looks up each fee with a tool, adds pairs with the calculator, compares with 30,000, and answers from real data. Takes several LLM calls.
Plan-and-Execute	Writes the plan first (look up three fees, form three pairs, compare, report), then executes it. Fewer model calls, and the plan can be checked before running.

Pattern	What happens
Reflection	Takes any of the above answers and reviews it: were all three pairs considered? Is the arithmetic right? Then issues a corrected, complete answer.

6. Key Terms

Term	Meaning
Orchestration loop	The code that repeatedly calls the model, runs tools, and feeds results back
Working memory	The message list for the current task
Long-term memory	Facts stored outside the conversation and retrieved when needed
System prompt	Developer instructions placed before the user's message
Tool description	The text the model reads when deciding whether to use a tool
Chain-of-Thought	Prompting the model to produce intermediate reasoning steps
Zero-shot CoT	Triggering that reasoning with an instruction rather than examples
Self-consistency	Sampling several reasoning paths and taking the most common answer
ReAct	Interleaving Thought, Action, and Observation
Trajectory	The full sequence of thoughts, actions, and observations in one run
Plan-and-Execute	Producing a complete plan first, then carrying it out
Replanning	Revising the plan when a step fails or new information appears
Reflection	Reviewing and revising one's own output
Error propagation	An early mistake carried forward into later steps

7. Further Reading

1. J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," NeurIPS 2022
<https://arxiv.org/abs/2201.11903>
2. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," ICLR 2023
<https://arxiv.org/abs/2210.03629>
3. The ReAct project page, with example trajectories
<https://react-lm.github.io/>
4. A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," NeurIPS 2023
<https://arxiv.org/abs/2303.17651>
5. N. Shinn et al., "Reflexion: Language Agents with Verbal Reinforcement Learning," NeurIPS 2023
<https://arxiv.org/abs/2303.11366>
6. B. Xu et al., "ReWOO: Decoupling Reasoning from Observations," 2023
<https://arxiv.org/abs/2305.18323>
7. Anthropic, "Building effective agents" (patterns for workflows and agents)
<https://www.anthropic.com/engineering/building-effective-agents>

Text book reading: Chip Huyen, AI Engineering (O'Reilly, 2025), the chapter covering RAG and agents, sections on agent planning and tool use.